

Incorporating Realistic Margin Constraints: A Data-Driven Deep Reinforcement Learning Framework for Advanced Portfolio Management

Jingyi Gu¹, Wenlu Du², Xinyun Zhao, A M Muntasir Rahman³, and Guiling Wang⁴, *Fellow, IEEE*

Abstract—In portfolio management via data-driven reinforcement learning techniques, existing approaches mainly focus on cash-only trading, overlooking potential financial data-related benefits and risks associated with margin trading. Integrating margin accounts and their constraints, particularly in short sale scenarios, presents a significant yet ignored opportunity for advanced financial data engineering solutions. To bridge this gap, we introduce Margin Trader, an innovative reinforcement learning framework for margin trading. Margin Trader incorporates margin accounts and constraints into a realistic trading environment via sophisticated data engineering to handle complex financial data, accommodating long and short positions. It aims to optimize profit maximization and risk management, through *Margin Adjustment Module* and *Maintenance Detection Module* to process trading data under real market. Margin Trader supports various Deep Reinforcement Learning (DRL) algorithms and allows traders to customize key settings, such as equity allocation, margin ratios, and maintenance requirements, utilizing knowledge management in adapting strategies to market conditions, individual preferences, and risk tolerance. Experiments, utilizing distinct RL algorithms and rolling-based testing periods, demonstrate that Margin Trader effectively learns profitable trading strategies and manages risks in both bullish and bearish markets, achieving superior performance with highest Sharpe ratio, underscoring critical role of data engineering in enhancing trading strategies and performance.

Index Terms—Financial data engineering, financial data mining, reinforcement learning, portfolio management.

I. INTRODUCTION

PORTFOLIO management plays a critical role in the dynamic landscape of the stock market. Investors curate a diverse selection of financial assets, including stocks, futures, bonds, foreign currency, and cryptocurrencies. These assets are periodically reallocated, guided by the overarching objective of maximizing capital growth over an extended time horizon while simultaneously minimizing risks [1], [2]. With the widespread

utilization of deep learning within the financial market [3], [4], [5], [6], [7], Deep Reinforcement Learning (DRL) [8] has emerged as a prominent and widely adopted approach for portfolio management, primarily owing to its inherent capacity for sequential decision-making [9], [10], [11]. In contrast to conventional investment strategies, DRL-based approaches have demonstrated considerable potential in adapting to dynamic market conditions and attaining superior returns [12], [13], [14], [15], [16], [17].

The majority of existing works using DRL have been confined to trading with their own cash, limiting the exploration of *margin accounts* that allow investors to amplify their exposure to the market and potentially enhance their returns through leveraging. By enabling both long and short positions, margin trading opens up a wide range of strategies, including hedging, short-selling, and arbitrage, which can help traders manage risk and capitalize on diverse market trends [18]. However, the potential for excessive leverage can amplify losses and introduce greater volatility into the portfolio. Proper risk management becomes critical when using margin trading. Margin accounts have constraints, such as initial and maintenance margins, to prevent excessive losses [19] by automatically triggering position closure when equity falls below a predefined threshold. Some studies assumed that borrowing stocks from brokers is unrestricted using a zero-investment strategy (i.e., the market value for long positions equals that of short positions) [20], ignoring that short sales in practice require margin accounts in reality. Furthermore, some works neglect the maintenance margin requirement [21], assuming traders can continue trading despite insufficient margin, with the expectation of recovering losses over time. This unrealistic assumption artificially inflates reported returns and underestimates the risk of forced liquidation. Therefore, the incorporation of margin accounts into RL-based portfolio management is still under-explored and it is essential for creating more realistic and practical strategies. Designing mechanisms to ensure compliance with constraints and regulate investors' risk is also a non-trivial task.

To fill these gaps, we develop Margin Trader, a foundational and flexible reinforcement learning framework tailored for margin trading in the stock market. Our framework goes beyond the cash-only trading approach by incorporating margin accounts and constraints via financial data engineering solutions, thereby allowing traders to leverage their positions in both long

Received 30 May 2024; revised 17 August 2025; accepted 28 May 2026. Date of publication 26 June 2026; date of current version 8 August 2026. Recommended for acceptance by J. Gao. (Jingyi Gu, Wenlu Du, Xinyun Zhao, and A M Muntasir Rahman contributed equally to this work.) (Corresponding author: Guiling Wang.)

Jingyi Gu, Xinyun Zhao, A M Muntasir Rahman, and Guiling Wang are with the New Jersey Institute of Technology, Newark, NJ 07102 USA (e-mail: jg95@njit.edu; xz43@njit.edu; ar238@njit.edu; gwang@njit.edu).

Wenlu Du is with the Skidmore College, Saratoga Springs, NY 12866 USA (e-mail: wdu@skidmore.edu).

Digital Object Identifier 10.1109/TKDE.2026.3701681

and short directions. This integration provides a more realistic trading environment. The primary objective of Margin Trader is to strike a delicate balance between profit maximization and risk management. To achieve it, we implement two key modules: the *Margin Adjustment Module* timely updates the buying power to ensure traders' maximum potential returns, and the *Maintenance Detection Module* protects the stability of the portfolio by prompt alerting traders when the margin level approaches critical points.

One of Margin Trader's key strengths lies in its exceptional flexibility. It allows traders to customize crucial settings for the trading environment. Traders can adapt their data-centric strategies to everchanging market conditions by fine-tuning equity allocation to long and short positions, manage risk tolerance levels by adjusting maintenance requirements, and cater to either conservative or aggressive trading styles based on their individual preferences by controlling margin ratios. This level of customization enables traders to identify more profitable opportunities while effectively mitigating unnecessary risk. Moreover, although this paper focuses on using stocks as trading securities for presentation, Margin Trader is versatile and can be extended to encompass various financial markets and instruments, such as futures and cryptocurrency markets. By appropriately defining the trading environment, setting relevant parameters, and adapting the RL algorithms, the framework can be tailored to suit the characteristics of different asset classes, making it applicable in a broader range of financial scenarios.

To comprehensively quantify the performance of Margin Trader, we implement Margin Trader in the portfolio pool of Dow Jones Industrial Average (DJIA) from 2020/05 to 2023/05, encompassing the COVID pandemic recovery in 2020 and market volatility in 2022. Margin Trader enables trading on both long and short positions and outperforms cash-only trading baselines by achieving the lowest volatility (0.163) while the highest Sharpe ratio (1.16) in both bullish and bearish markets. Even in the long position alone, it exhibits a remarkable three-fold return (1.234) compared with DJIA index (0.437) and the highest Calmar ratio (1.199). To evaluate the Margin Trader with various market conditions to demonstrate its generalization and robustness, Margin Trader is also assessed under rolling-based testing periods which often lacking in existing works for portfolio management evaluation. The superior performance highlights Margin Trader's effective learning of profitable and safer trading strategies, making it a promising option for traders seeking to optimize their trading approaches. Our contributions are summarized as follows:

- 1) We propose Margin Trader, the first data-driven framework that integrates margin account and constraints into reinforcement learning for portfolio management.
- 2) Margin Trader provides versatility, enabling traders to customize various settings to align with market conditions, risk tolerance, and individual preferences.
- 3) The experiment in rolling-based testing periods showcase that Margin Trader effectively learns trading strategies that employ margin to amplify profits and hedge risks simultaneously, both in bullish and bearish market conditions.

II. RELATED WORKS

In this section, we present a review of existing RL-based portfolio management frameworks. Two main lines of study have emerged in this field. The first line focuses on specific model structures, often incorporating tailored data or architectures to address specific challenges (e.g., transaction costs and asset interrelationships). Another line of research aims to develop generalized frameworks that offer flexibility across different market environments and financial tasks.

A. Specific Model Designs for Portfolio Management

1) *Long-Only Strategies*: Notably, the study [22] presented an RL framework using an Ensemble of Identical Independent Evaluators (EIIE) topology, considering transaction costs and previous period weights. Alpha Stock [23] is proposed as an RL-based model with a Cross-Asset Attention Network to model interrelationships among assets and a portfolio generator, and is optimized towards the Sharpe Ratio in a return-risk-balanced way. HRPm [24] addressed commission fee and non-zero price slippage using a deep hierarchical RL method. The high-level policy learns to generate portfolio weights to maximize profit, while the low-level policy minimizes trading costs. Lately, Smart Trader [25] integrates Encoder-Decoder into the Geometric Brownian Motion process to model unobservable and complicated stochastic processes of price time series, and utilizes Graph Attention Network (GAT) for portfolio weight calculation.

2) *Long & Short Strategies*: Unlike the studies above that consider only the long position, several studies have incorporated both long and short positions in their research. iRDPG [26] introduced the Recurrent Deterministic Policy Gradient with Imitative Learning to address noisy high-frequency financial data and balance exploration and exploitation. Although they include short positions in their trading actions, there is a lack of detailed explanation regarding the short setting, which is crucial in real-world trading. Similarly, other studies, such as Deep Trader and Meta Trader [20], [27], also involve both long and short operations. However, a major limitation of these studies is the strong assumption of a zero market, where the long market value is considered equal to the short market value. These studies typically involve selling stocks in a long position, borrowing stocks initially, mortgaging stocks based on the short position, and making purchases based on the long proportion. Additionally, RAT [28] explored margin trading using a transformer-based framework but ignored that margin trading requires investors to have sufficient equity as the deposit. Huang et al. [21] added short and arbitrage mechanisms based on EIIE but neglected to enforce real-world constraints such as maintenance margin requirement. In practice, positions are forced to be closed when equity falls below a certain threshold, preventing further trading. However, their work allows continued trading, recovering losses, and even earning profits despite insufficient margin. This unrealistic assumption leads to exaggerated returns. Some high-frequency trading strategies [13] also incorporate short sales, but they highly rely on limit order book data, including bid-ask prices and order quantities at multiple price levels, since

high-frequency trading operates at minute-level intervals and requires real-time market liquidity and order flow information.

Therefore, it's essential to incorporate real-world constraints to provide a more practical solution. Since these works have unrealistic assumptions that may lead to exaggerated profits and data-specific requirements, direct comparisons between these works and Margin Trader would not be fair.

B. Generalized RL Frameworks for Portfolio Management

In contrast to specific models, which require tailored data, specialized architectures, and training for each use case, generalized RL-based frameworks provide greater flexibility across diverse market environments and financial tasks. Notable examples include the FinRL and FinRL-Meta library [29], which offer modular environments for training agents across various financial markets. These frameworks offer multiple market environments, reproducing popular benchmarks, and cloud deployment for visualization and assessment. With data APIs from different market platforms, it enables training agents for various financial tasks. However, despite the broad applicability, it primarily focuses on long positions, motivating our work to extend it by incorporating margin trading and enforcing constraints to more accurately mimic real-world trading procedures.

III. PRELIMINARY

A. Trading With a Margin Account Allowing Both Long and Short Positions

This paper introduces a more realistic trading setting, allowing trading on a margin account with both long and short positions. The portfolio pool contains equity positions, enabling traders to either long a stock when expecting price increases or short a stock to profit from anticipated price declines. Utilizing a margin account, trading is not limited to cash; instead, traders can borrow funds or stocks from brokers, increasing their trading capacity [19].

The **initial margin requirement**, typically set at 50%, necessitates a deposit into the margin account. In a long position, with deposit D^l , traders can borrow up to D^l in cash as a **loan** and purchase stocks valued at a maximum of $2D^l$. In a short position, with deposit D^s , they can borrow stocks valued at $2D^s$ from brokers, resulting in a total **credit balance** of $3D^s$. A 50% margin requirement implies a leverage of 2, amplifying any price fluctuation's impact on the portfolio value. If the equity falls below the **maintenance margin requirement** (usually 40% of the holding market value), a **margin call** is triggered, requiring additional cash deposits or position closure. As the margin account's value increases, **additional buying power** is obtained, allowing further borrowing without additional cash deposits.

The timeline is divided into **holding periods**, during which traders adjust their portfolios based on price fluctuations. At the beginning of the holding period t , traders hold a portfolio of N stocks with $n_{i,t}$ shares for stock i at a price of $p_{i,t}$. Throughout the holding period, the fluctuation in prices leads to the change in equity value from e_t to e_{t+1} . At the end of the holding period

$t + 1$, traders make decisions regarding buying or selling stocks to adjust the portfolio accordingly. If a margin call occurs at the first time, new trading is forbidden; for the second time, the position will be closed. The details about how to handle margin calls are presented in Sections IV-C2 and IV-C3.

B. Assumptions

In line with numerous existing literatures [22], [28], [29], we adopt the following assumptions.

- 1) *Zero slippage*: We assume that the market exhibits high liquidity, enabling trades to be executed instantly at the specified order price. Consequently, the actual trade price aligns precisely with the price indicated in the order.
- 2) *Zero market impact*: Our investment capital remains relatively small, exerting minimal influence on the market. As a result, our trading activities do not disrupt the supply and demand dynamics or significantly affect market prices.
- 3) *Transaction cost*: Each trade incurs an additional fee known as transaction cost, denoted as c for either buying or selling shares of stocks.

IV. MARGIN TRADER

Margin Trader is tailored for real-world trading scenarios involving margin accounts with both long and short operations. It harnesses the power of Reinforcement Learning to equip its agent with the ability to effectively observe and interact with the dynamic trading environment, enabling the learning and development of robust trading strategies. We delve into the core functionalities and modules of Margin Trader in this section. Fig. 1 provides a big picture of Margin Trader.

A. Customizable Settings and Notations

Margin Trader provides traders with customizable settings, allowing flexibility to adapt to prevailing market conditions and specific requirements:

- 1) *Ratio of long position to equity (r)*: Traders can customize the allocation of the initial capital value to long and short positions at t_0 , according to their preferences. This ratio, denoted by r , determines the distribution of equity between the two positions and remains fixed during trading. For a given ratio r for the long position, the initial cash in the long position is $e^l = er$, while the initial cash in the short position is $e^s = e(1 - r)$.
- 2) *Initial margin requirement (k)*: The initial margin requirement, denoted by k , governs the extent to which traders can leverage their equity. When allocating an equity amount of e^l to the long position, traders can borrow up to $(k^l - 1)e^l$ in cash and trade stocks with a market value of up to $k^l e^l$. Similarly, in the short position, with a deposit of e^s , traders can borrow stocks from brokers with a market value of up to $k^s e^s$. For our experiments, we set $k^l = k^s = 2$, resulting in a leverage ratio of 2.
- 3) In addition to transaction cost c , we denote the initial equity value as e . That means, at the start of trading, the trading account is initialized with a cash amount of e .

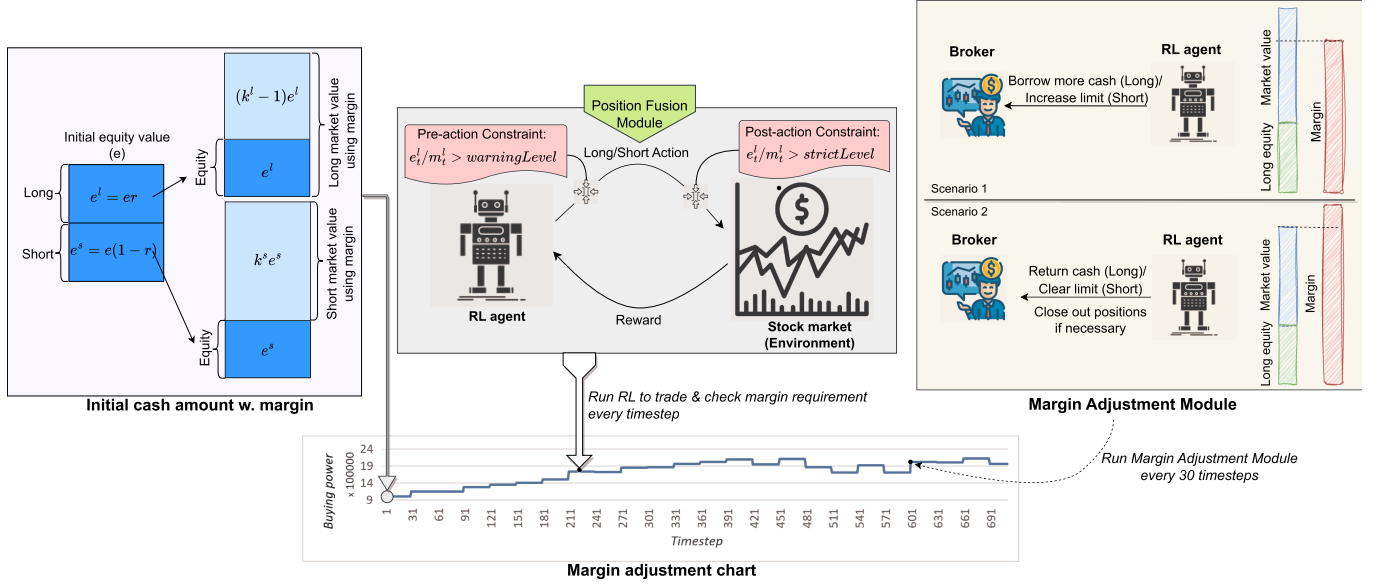


Fig. 1. The interaction process between agent and environment in the Margin Trader.

B. Reinforcement Learning Framework

In this paper, we model the margin trading scenario as a Markov Decision Process (MDP) to apply reinforcement learning (RL). The RL agent acquires decision-making capabilities by interacting with an environment represented as an MDP. The MDP consists of several components, including state space S , action space A , transition probabilities P , a reward function R , and a discount factor γ . At each time step, the agent observes a state S_t , selects an action A_t based on a policy π , and transitions to a new state S_{t+1} according to the transition probability $T(S_{t+1}|S_t, A_t)$. The agent receives a reward R_t for its action. The action-value function Q^π evaluates the quality of taking a specific action in a given state. It is expressed as the expected cumulative reward:

$$Q^\pi(s, a) = \mathbb{E}[\sum_{i=t}^{\infty} \gamma^i r_{t+i} | s_t = s, a_t = a] \quad (1)$$

The goal of the RL agent is to learn the optimal policy π^* that maximizes the action-value function Q^π .

1) *State*: The state representation in Margin Trader is designed to capture both the agent's equity condition and the macro market environment at the current time step t , enabling effective portfolio tracking and management. It is represented by the state vector $S_t = [b_t^l, l_t^l, e_t^l, l_t^s, b_t^s, e_t^s, \mathbf{p}_t, \mathbf{n}_t, \mathbf{t}_t]$, where each component carries specific information:

- 1) Available cash balance in long position b_t^l : the remaining cash in the long position available for purchasing stocks.
- 2) Loan l_t^l : the amount of cash borrowed by the agent.
- 3) Long equity e_t^l : the equity held by the agent in the long position. It is the value of the holding asset (available cash and holding long market value m_t^l) after returning borrowed loan:

$$e_t^l = b_t^l + m_t^l - l_t^l \quad (2)$$

- 4) Available limit in short position l_t^s : the remaining market value that can be shorted.
- 5) Credit balance b_t^s : the sum of the initial limit of borrowed funds and initial deposit.
- 6) Short equity e_t^s : the equity held by the agent in the short position. It is calculated by subtracting credit balance from the available limit and holding short market value m_t^s :

$$e_t^s = b_t^s - m_t^s - l_t^s \quad (3)$$

- 7) Close prices $\mathbf{p}_t = \{p_{i,t}\} \in \mathbb{R}^N$: close price for each stock i .
- 8) Holding shares $\mathbf{n}_t = \{n_{i,t}\} \in \mathbb{R}^N$: the number of shares held by the agent for each stock i . The value of $n_{i,t}$ indicates the agent's position in the stock i . A positive value shows that it is in a long position. A negative value signifies that it is shorted. A value of 0 implies that it is not traded in either long or short positions.
- 9) Technical indicators $\mathbf{n}_t = \{\mathbf{n}_{i,t}\} \in \mathbb{R}^{4 \times N}$: the technical indicators associated with each stock. Following the specific settings in [30], $\mathbf{n}_{i,t}$ includes Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI), Commodity Channel Index (CCI), and Average Directional Index (ADX) for the stock i .

In the reset step, the agent allocates the initial equity to long and short positions. In the long position, the agent deposits the initial equity e^l and borrows a loan of $l_0 = (k^l - 1)e^l$. Hence the available cash for stock purchases is $b_0^l = k^l e^l$. In the short position, the agent provides an initial deposit of e^s and the initial limit for short-selling is $l_0^s = k^s e^s$, hence credit balance reaches $b_0^s = (k^s + 1)e^s$. As no stocks are initially held, all elements in $\mathbf{n}_0$ are initialized to 0.

2) *Action*: $A_t = [\mathbf{a}_t^l, \mathbf{a}_t^s, \mathbf{a}_t^r]$ consists of actions for long position $\mathbf{a}_t^l = \{a_{i,t}^l\} \in \mathbb{R}^N$, short position $\mathbf{a}_t^s = \{a_{i,t}^s\} \in \mathbb{R}^N$ and a single action for long/short ratio changes $\mathbf{a}_t^r = \{a_{i,t}^r\} \in \mathbb{R}^N$.

For each stock i , the agent chooses an integer from a discrete space $A^l = \{-h, \dots, 0, \dots, h\}$ and $A^s = \{-h, \dots, 0, \dots, h\}$ as the action defined as the number of shares to trade. A positive value for $a_{i,t}^l$ or $a_{i,t}^s$ represents the number of shares to buy in long or buy in short, and a negative value implies to sell the stock in long or short. h is a parameter to control the maximal quantity of shares that can be traded for a single stock at each step. The action to change the long and short ratio is a continuous sequence of natural numbers in the range between $-h^r$ and h^r : $A^r = \{-h^r, \dots, 0, \dots, h^r\}$. A positive number indicates an increase in the proportion of equity in the long position, and conversely, a negative number indicates an increase in the proportion of the short position. The action representation enables the agent to make discrete decisions, providing flexible control over the trading strategy and maintaining interpretability.

3) *Reward*: We define the reward function by profit and risk with reward shaping:

$$R_t(S_t, A_t, S_{t+1}) = \lambda_1 profit_t + \lambda_2 risk_t \quad (4)$$

The profit is determined by the difference in equity between two consecutive time steps: $e_{t+1} - e_t$. It can be decomposed into two parts: transaction cost incurred due to trading actions and price changes in the macro market. First, the agent adjusts the portfolio based on the action A_t . The quantity of holding shares is updated to $\mathbf{n}_t + \mathbf{a}_t^l + \mathbf{a}_t^s$. The traded market values are given by $\sum_i |a_{i,t}^l| p_{i,t}$ for long position and $\sum_i |a_{i,t}^s| p_{i,t}$ for short position, which impact the available cash balance b_t^l and available limit in short sale l_t^s , respectively. The transaction cost associated with the trading activities affects the long equity, short equity, available cash balance, and credit balance accordingly. As the time step progresses to $t + 1$, the price fluctuates to $\mathbf{p}_{t+1}$, resulting in the updates to the holding market value in long and short positions:

$$m_{t+1}^l = \sum_{i, n_{i,t+1} > 0} n_{i,t+1} p_{i,t+1} \quad (5)$$

$$m_{t+1}^s = \sum_{i, n_{i,t+1} < 0} |n_{i,t+1}| p_{i,t+1} \quad (6)$$

The available cash balance, loan, available limit in the short sale, and credit balance remain unchanged. Consequently, the next state S_{t+1} can be observed and the profit can be calculated:

$$b_{t+1}^l = b_t^l - \mathbf{a}_t^{l\top} \mathbf{p}_t - c \sum_i |a_{i,t}^l| p_{i,t} \quad (7)$$

$$e_{t+1}^l = b_{t+1}^l + m_{t+1}^l - l_{t+1}^l \quad (8)$$

$$l_{t+1}^l = l_t^l \quad (9)$$

$$b_{t+1}^s = b_t^s - c \sum_i |a_{i,t}^s| p_{i,t} \quad (10)$$

$$e_{t+1}^s = b_{t+1}^s - l_{t+1}^s - m_{t+1}^s \quad (11)$$

$$l_{t+1}^s = l_t^s + \mathbf{a}_t^{s\top} \mathbf{p}_t \quad (12)$$

$$\mathbf{n}_{t+1} = \mathbf{n}_t + \mathbf{a}_t^l + \mathbf{a}_t^s \quad (13)$$

$$profit_t = e_{t+1} - e_t = (e_{t+1}^l + e_{t+1}^s) - (e_t^l + e_t^s) \quad (14)$$

The risk component of the reward function is represented by the Sharpe ratio [31], a widely used metric for assessing the risk-adjusted return of a portfolio. We calculate the Sharpe ratio based on the equity values of the most recent five steps:

$$SR = \frac{AR_p - AR_f}{\sigma_p} \quad (15)$$

where AR_p is the average return of the portfolio, AR_f is the risk-free rate, and σ_p is the standard deviation of the return of the portfolio.

The reward function provides a comprehensive evaluation of the agent's trading performance. It incorporates actual profits, accounting for transaction costs and the market value changes, while also factoring in the risk profile of the portfolio.

4) *DRL Algorithms*: Margin Trader is designed to accommodate a variety of Deep Reinforcement Learning (DRL) algorithms to facilitate effective learning and interaction with the trading environment. There are three main categories of algorithms widely used for stock trading: policy-based [32], value-based [33], [34], and actor-critic-based algorithms [35], [36]. In Margin Trader, we choose actor-critic models due to their combination of value-based and policy-based approaches. Specifically, an actor-critic algorithm employs both a critic and an actor. The critic estimates the action-value function, while the actor learns the policy distribution in the direction guided by the critic. Both actor and critic are parameterized with neural networks. We apply four popular actor-critic models, Advantage Actor-Critic (A2C) [37], Proximal Policy Optimization (PPO) [38], Deep Deterministic Policy Gradient (DDPG) [39], and Soft Actor-Critic (SAC) [40].

- 1) A2C is a foundational actor-critic algorithm that uses advantage estimation to focus on the relative value of actions, improving policy updates and reducing variance. Its synchronous training collects experiences across multiple environments, ensuring computational efficiency. It is best suited for small to medium portfolios in volatile markets, where its balance of exploration and exploitation effectively handles noisy and dynamic scenarios.
- 2) PPO extends the actor-critic framework by introducing a clipped objective function to restrict large policy updates, ensuring stability and avoiding policy collapse. Its efficiency and reliability make it a preferred choice for long-term investment strategies in stable markets or institutional portfolio management, where consistent returns and robust performance are essential.
- 3) DDPG is designed for continuous action spaces, enabling precise control over asset allocation. By employing deterministic policy updates with experience replay and target networks, DDPG provides stable learning. This makes it highly effective in stable market environments that require fine-grained adjustments, such as managing large and diversified portfolios.
- 4) SAC utilizes entropy maximization to encourage robustness, making it ideal for highly volatile or unpredictable markets. Its twin Q-networks and clipped double-Q learning minimize value estimation biases, while its off-policy design efficiently reuses past experiences. SAC excels

in high-frequency trading and emerging markets, where adaptability and resilience under uncertainty are critical.

Each algorithm offers distinct advantages. Selecting the appropriate algorithm based on portfolio requirements and market conditions enhances overall portfolio performance. In Margin Trader, the actions generated from the actor network are normalized in the range of $[-1, 1]$ and then scaled by h to determine the number of shares to trade.

C. Optimization and Margin Handling

Margin Trader incorporates modules to optimize trading and ensure compliance with margin requirements. These modules focus on enhancing financial stability, minimizing risks, and maximizing the efficient use of leverage for higher profit.

1) *Position Fusion Module*: The action output from the RL agent can be long and short on the same stock. To manage this conflict, we enforce a rule that stocks can only be traded either in a long position or a short position, but not both simultaneously. Once the action A_t is generated, it is passed through a *Position Fusion Module*. For each stock i , two actions $a_{i,t}^l$ and $a_{i,t}^s$ are produced, corresponding to long and short positions respectively. The signs of two actions reflect the market sentiment and are consistent between two positions. For instance, a negative value in the long position indicates selling the stock to avoid potential losses, while in the short position, it represents selling the stock to profit from the anticipated price declines. This alignment ensures that the actions correspond to the intended market direction for each position. Therefore, we sum them up to obtain the net quantity $a'_{i,t} = a_{i,t}^l + a_{i,t}^s$ and assign it to the current holding position. If $n_{i,t} > 0$, indicating the stock is already held in a long position, then $a_{i,t}^l = a'_{i,t}$ and $a_{i,t}^s = 0$. Conversely, if $n_{i,t} < 0$, indicating a short position, then $a_{i,t}^s = a'_{i,t}$ and $a_{i,t}^l = 0$. In cases where the stock has not been traded before, and the agent intends to initiate a trade in the current time step, the positive net quantity is assigned to the long position, while the negative quantity is assigned to the short position. After passing through the *Position Fusion Module*, the updated action is executed by the agent to make adjustments to the portfolio.

2) *Margin and Buying Power Adjustment*: This module is designed to regularly update the buying power, allowing traders to leverage their positions to maximize potential returns. The maximal amount of loan that can be borrowed and the maximal market value that can be shorted in the Margin Trader are initially determined based on the initial equity e . However, as the market fluctuates and the equity value changes, it becomes necessary to update these values to ensure they align with the agent's current holding equity. To facilitate it, we design a *Margin Adjustment Module*. It is activated for periodic evaluation and updates of positions.

For the long position, the module begins by comparing the borrowed loan l_t^l with the maximal amount of loan that can be borrowed based on the current equity $(k^l - 1)e_t^l$. This comparison leads to two different scenarios.

- 1) If the borrowed loan is lower than the maximal limit, it indicates that the agent has earned a profit during this period. The increased equity provides additional buying

power, prompting the agent to enlarge the loan. By borrowing more cash, the agent leverages its position further and aligns itself with the desired margin ratio:

$$l_t^l = (k^l - 1)e_t^l \quad (16)$$

$$b_t^l = k^l e_t^l - m_t^l \quad (17)$$

- 2) Conversely, if the borrowed loan exceeds the maximal limit, it implies that the agent has experienced a loss in their holdings. In this situation, the agent must take appropriate actions to reduce the loan and align it with the initial margin requirement $(k^l - 1)e_t^l$. This process involves two steps:

- If the agent holds available cash, then return it to reduce the borrowed funds and bring the loan in line with the initial margin requirement.
- If returning available cash is insufficient to meet the margin requirement, the agent is forced to progressively close out positions to generate necessary funds. The closure process starts with the stock with the lowest holding market value, prioritizing the liquidation of the least significant portfolio positions, and ensuring the loan is reduced to the required level ultimately. When dealing with the last asset in this process, two options arise: (a) partial closure involves liquidating a minimum portion of the position or shares to satisfy the margin requirement, or (b) complete closure entails closing out all positions for this asset to ensure compliance with the margin requirement.

Similarly, for the short position, the module compares the current credit balance b_t^s with the maximal credit balance based on current equity $(k^s + 1)e_t^s$.

- 1) If the current credit balance is lower than the maximal limit, the agent is permitted to sell more stocks in the short sale. Consequently, the credit balance is increased to $(k^s + 1)e_t^s$, and the available limit for short sale l_t^s is adjusted to fill the difference:

$$b_t^s = (k^s + 1)e_t^s \quad (18)$$

$$l_t^s = k^s e_t^s - m_t^s \quad (19)$$

- 2) If the current credit balance exceeds the maximum limit, the module takes steps to reduce it:

- The agent clears the available limit l_t^s to decrease the credit balance.
- If it is not sufficient, the agent is forced to progressively close out the positions by buying back the stocks and returning them to the brokers. The process starts from the stocks with the lowest shorted market value and continues until the credit balance is reduced to meet the necessary requirements. Liquidation of the last asset also has two options: partial closure or complete closure.

The update cycle is flexible, allowing updates every n_{count} ($n_{count} \in \mathbb{Z}^+$) days, weeks, months, or years. Since trades operate in real-world time, the module's update schedule synchronizes with the actual calendar to maintain alignment with real-world dates. For example, if the model is set to update every

two months, there would naturally be fewer trading dates during February's cycle due to its fewer days compared to other months.

By incorporating the *Margin Adjustment Module*, Margin Trader dynamically adjusts the loan and credit balance in long and short positions to maintain alignment with the agent's current equity. This adaptive approach optimizes the leveraging of profitable periods while mitigating risk during downturns, mimicking real-world trading environments, and contributing to the overall effectiveness of the trading strategy.

3) *Margin Maintenance Detection Module*: To ensure compliance with the maintenance margin requirement in the margin account, which mandates that the ratio of equity to market value in a position (e_t^l/m_t^l or e_t^s/m_t^s) must exceed a certain threshold, we implement *Maintenance Detection Module* equipped with a pre-action constraint and post-action constraint to continuously monitor the margin levels and trigger necessary actions if it is not met, safeguarding the portfolio's safety. Specifically, it establishes two levels associated with the maintenance margin requirement: the warning level and the strict level.

- 1) For the pre-action constraints, the warning level is employed to promptly notify the agent before purchasing long stocks or selling short positions. If the maintenance margin is detected to fall below the requirement level, additional trading actions on this stock are forced to halt, and a warning message is displayed. It warns the agent that the margin level is approaching a critical point and it should take action.
- 2) For the post-action constraints, the strict level is applied after all actions A_t for both long and short positions have been executed at each step. If a margin call is triggered for any position, the *Margin Adjustment Module* is activated. The portfolio is required to be reallocated, specifically reducing the loan limit or credit balance limit, to meet the necessary maintenance level. It mitigates the potential risks associated with inadequate equity coverage.
- 4) *Maximum Quantity of Shares to Trade*: When the agent executes buying or selling actions for stocks, the actual quantity of shares that can be traded is subject to certain constraints and adherence to available resources.

- 1) For the stocks to be bought in the long position, the market value of the trade cannot exceed the available cash. It ensures that the cash is not depleted beyond the limit.

$$a_{i,t}^l = \min \left(a_{i,t}^l, \frac{b_t^l}{p_{i,t}(1+c)} \right), \quad \text{if } a_{i,t}^l > 0 \quad (20)$$

- 2) For the stocks to be sold in the short position, the market value of the trade cannot surpass the available limit for the short sale.

$$a_{i,t}^s = -\min \left(|a_{i,t}^s|, \frac{l_t^s}{p_{i,t}(1+c)} \right), \quad \text{if } a_{i,t}^s < 0 \quad (21)$$

- 3) For the stocks to be sold in the long position or bought back in the short position, the quantity of shares traded cannot exceed the current holding shares.

$$a_{i,t}^l = -\min(|a_{i,t}^l|, n_{i,t}), \quad \text{if } a_{i,t}^l < 0 \quad (22)$$

$$a_{i,t}^s = \min(a_{i,t}^s, |n_{i,t}|), \quad \text{if } a_{i,t}^s > 0 \quad (23)$$

V. EXPERIMENTS

A. Portfolio Pool

We choose Dow Jones Industrial Average (DJIA) as our portfolio pool and execute trading actions among the stocks of DJIA.¹ We follow the data cleaning and feature engineering procedure in FinRL-Meta [29]. The stock data and technical indicators can be collected from Yahoo Finance. The stock data is divided along with the temporal dimension into three datasets for training, tuning parameters, and testing. The model is trained on the period of 2009/01 to 2018/12; the hyperparameters are tuned based on the validation set from 2019/01 to 2020/04; the performance is evaluated over the testing period from 2020/05 to 2023/05, encompassing over 700 trading days. The testing period includes the recovery period from the COVID pandemic in 2020 and a notable phase of market volatility experienced in 2022. Our code is publicly available.²

B. Baselines

We compare the performance of Margin Trader with three agents, DJIA, FinRL, and DeepTrader.

- 1) *DJIA*: The DJIA agent tracks the DJIA index to trade.
- 2) *FinRL* [30]: Chosen as one of the baselines since it is a state-of-the-art benchmark to apply DRL algorithms in portfolio management. Its trading environment is limited to the long position only and does not include margin trading capabilities.
- 3) *Deep Trader* [20]: A DRL framework for portfolio management that incorporates market condition embeddings to balance risk and return, with an asset scoring unit that ranks assets based on temporal and spatial price dependencies, and a market scoring unit that adjusts the long-short proportion according to macro indicators. DeepTrader was originally designed for monthly trading, with its investment policy optimized end-to-end using policy gradient methods. In our implementation, we adapt DeepTrader to daily trading mode to match our settings and ensure a fair comparison.
- 4) *Margin Trader*: Enables both long and short positions. It is necessary to compare their impact and performance under different market conditions. Considering that the long-term market trend generally exhibits an upward trajectory with short-term fluctuations, we implement two distinct modes for the position settings: margin on long position, and margin on both long and short positions.

For a fair comparison, four popular DRL algorithms, A2C, PPO, DDPG, and SAC, are applied in FinRL, and the two modes of Margin Trader.

C. Evaluation Metrics

We follow [29] to select six metrics to comprehensively evaluate the performance of the framework.

¹Our security pool can be easily extended to other stock indices or other financial products, e.g., cryptocurrency.

²<https://github.com/JingyiGu/Margin-Trader>

TABLE I
NUMERICAL RESULTS OF FOUR AGENTS (DJI, DEEP TRADER FINRL, AND MARGIN TRADER) AND DIFFERENT POSITION SETTINGS IMPLEMENTED BY ACTOR-CRITIC ALGORITHMS. THE BEST PERFORMANCE IS MARKED IN BOLD. THE FIRST FOUR METRICS ARE EXPRESSED AS PERCENTAGES.

Agent	Model	Accumulated Return	Annual Return	Annual Volatility	Maximum Drawdown	Sharpe Ratio	Calmar Ratio
DJI		43.7%	12.9%	17.3%	-21.9%	0.790	0.587
Deep Trader		-13.39%	0.24%	31.9%	-43.2%	0.008	0.006
FinRL (Long)	A2C	38.3%	11.5%	18.4%	-24.2%	0.684	0.474
	PPO	35.2%	10.6%	17.0%	-13.9%	0.680	0.764
	DDPG	60.6%	17.2%	18.4%	-23.4%	0.956	0.733
	SAC	54.4%	15.6%	18.2%	-19.5%	0.890	0.801
Margin Trader (Long & Short)	A2C	66.3%	18.5%	25.6%	-26.1%	0.793	0.710
	PPO	69.0%	19.2%	16.3%	-18.2%	1.160	1.057
	DDPG	70.8%	20.9%	27.2%	-25.7%	0.835	0.812
	SAC	65.1%	18.3%	21.2%	-19.2%	0.901	0.950
Margin Trader (Long)	A2C	118.9%	30.0%	29.3%	-27.1%	1.044	1.106
	PPO	123.4%	30.9%	30.1%	-25.8%	1.046	1.199
	DDPG	110.0%	30.1%	30.7%	-32.9%	1.012	0.915
	SAC	113.4%	28.9%	29.4%	-31.5%	1.011	0.916

- 1) **Accumulated Return R** : total return achieved over the testing period.
- 2) **Annual Return AR** : the percentage change in equity over a one-year period, widely used to assess long-term performance.
- 3) **Annual Volatility VOL** : the standard deviation of annualized return. A higher value indicates higher fluctuations in the return.
- 4) **Maximum Drawdown MDD** : the maximum loss from the peak to trough during the trading period. A lower value indicates a volatile downside movement.
- 5) **Sharpe Ratio SR** : a risk-adjusted metric to evaluate the excess return relative to its risk. A higher SR implies a more favorable risk-to-reward strategy.
- 6) **Calmar Ratio CR** : a risk-adjusted performance measuring the return relative to its maximum loss. A higher CR shows a better performance.

To summarize, R and AR evaluate the profit, VOL and MDD measure volatility and loss, SR and CR indicate both risk and return.

D. Implementation

For the parameters in the trading environment: the transaction cost c is set to 0.1% of traded market value; the λ_1 in the reward function is set to 0.00001, and λ_2 is tuned in the range of {0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05}; in *Maintenance Detection Module*, the warning level is set to 40% and strict level is set to 30%; the maximal quantity of shares h is set to 100 for FinRL (long) and Margin Trader in long, it is set to 50 for Margin Trader in long & short, considering that it reaches 100 after *Position Fusion Module*.

The actor-critic algorithms are implemented by stable baselines3 [41], and MLP policy is used as the network in actors and critics. We follow the FinRL framework to tune the hyperparameters of DRL algorithms. Table II shows the range of hyperparameters for tuning during the training process.

TABLE II
HYPERPARAMETERS ON THE MARGIN TRADER AND FINRL

Model	Hyperparameter	Range
A2C	Entropy coefficient	{0, 0.005, 0.01}
	Number of steps per update	{5, 10, 15}
	Learning rate	[0.00001, 0.005]
PPO	Entropy coefficient	{0, 0.005, 0.01}
	Batch size	{64, 128, 256}
	Learning rate	[0.00001, 0.005]
DDPG	Buffer size	{50000, 100000}
	Batch size	{64, 128, 256}
	Learning rate	[0.00001, 0.005]
SAC	Buffer size	{50000, 100000}
	Entropy coefficient	{0, 0.01, 0.1}
	Batch size	{32, 64, 128}
	Learning rate	[0.00001, 0.005]
	Warm-up steps	{100, 500}

E. Results Comparison

We initialize each agent with an initial equity of \$100,000. FinRL trades up to a market value of \$100,000 exclusively in the long position. Margin Trader on the long position allocates its entire initial equity to the long market, thereby allowing trades with a maximum market value of \$200,000. In the case of margin on both long and short positions, half of the equity is allocated to each position, enabling trades with a maximum market value of \$100,000 for each position.

Table I presents the numerical results. Fig. 2 displays the equity change of Margin Trader, Deep Trader, and FinRL over the testing period. We have the following key observations.

- 1) Margin Trader on the long position achieves the highest final equity. Its annualized return is twice that of FinRL on long positions only, demonstrating that trading strategies involving margin accounts can effectively earn more profit

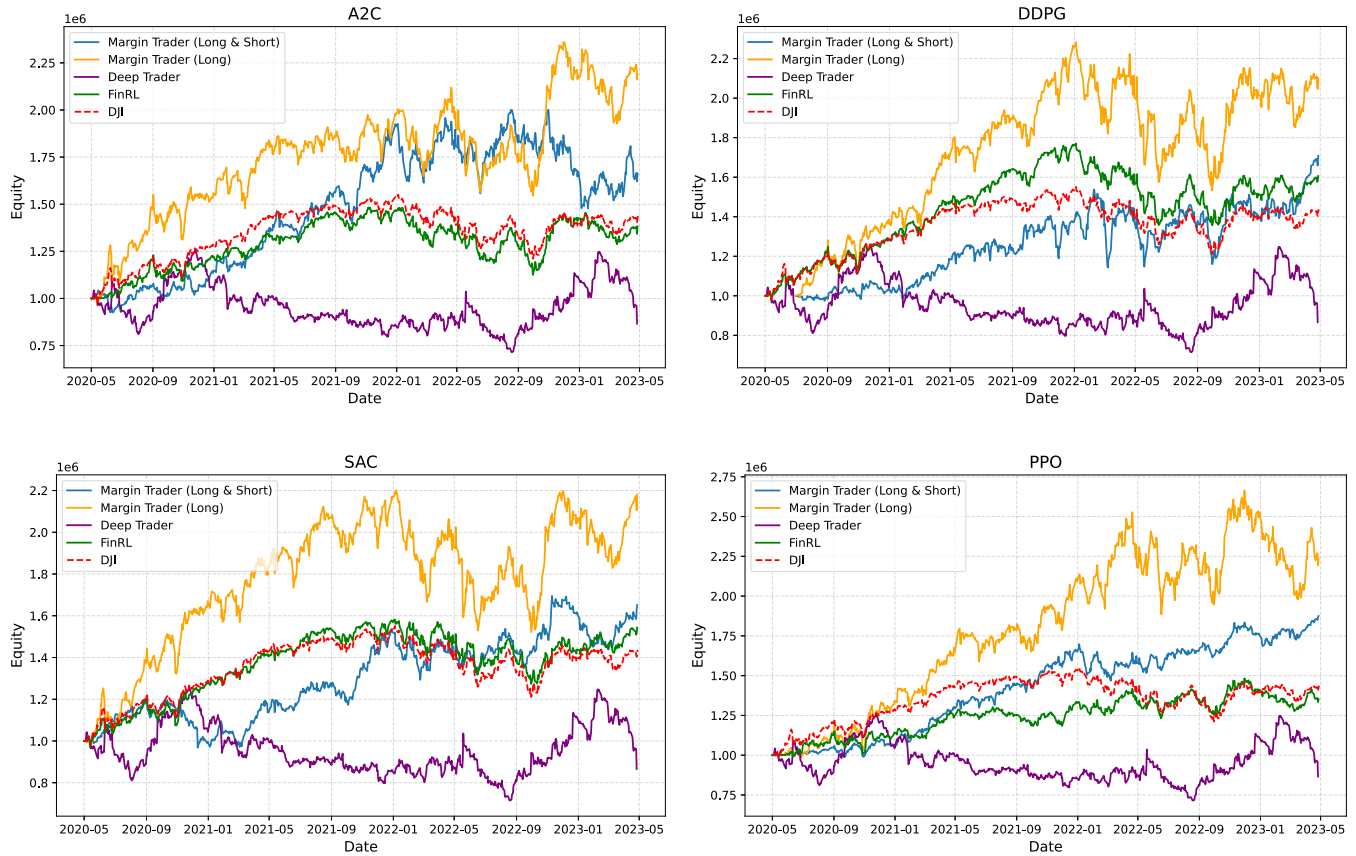


Fig. 2. The equity changes of DJI, FinRL, deep trader, and margin trader.

than trading with cash only, particularly in the bullish market from 2020/07 to 2022/01. In contrast, FinRL, which trades with its own cash in long positions, essentially follows the trend of DJIA agent. DeepTrader, however, records the lowest annualized return and delivers the worst performance on our dataset, highlighting its inability to adapt effectively in this setting.

- 2) During the volatile period, especially 2022/01 to 2022/10, DJIA, FinRL, Deep Trader, and Margin Trade in long positions show decreasing trends. However, Margin Trading in long & short positions exhibits an upward trend, particularly with SAC and PPO. A2C shows fluctuating equities in both modes of Margin Trader, but it does not lose many positions due to the help of maintenance margin constraints. This finding validates that incorporating short positions during volatile periods can hedge the risk and guarantee profit. However, this strategy is not robust in a bullish market since half of the equity is allocated into short positions which makes it difficult to profit from price increases.
- 3) Among different types of DRL algorithms, PPO consistently provides the best trading strategies for all positions regarding all metrics, regardless of bullish and bearish market conditions.
- 4) The two different modes of Margin Trader demonstrate the importance of selecting positions based on dynamic

market conditions. The long position outperforms during the first half of the period in a bullish market, while the short position makes a profit during the second half of the period in a bearish market. Properly choosing allocation ratios for long and short positions, as well as margin ratio, can magnify the profit and hedge the risk simultaneously. This flexibility in Margin Trader enables users to adapt their strategies according to market conditions and individual preferences.

- 5) DeepTrader, in our daily trading implementation, showed relatively fair performance in the early stage compared to Margin Trader and other baselines. However, its performance gradually decreased and collapsed into a -13.39% loss by the end of the testing period. The drawdown (-43.2%) indicates severe downside risk. We also tested DeepTrader in the monthly trading mode, where it can perform relatively well if trades are less frequent. Our Margin Trader outperforms DeepTrader across all DRL algorithms, which we attribute to several factors: (i) DeepTrader uses a vanilla policy gradient for optimization, which may be less sample-efficient than modern DRL algorithms such as PPO and SAC; (ii) its design is more suited for long-term, infrequent rebalancing, leading to slower adaptation in rapidly changing markets; and (iii) DeepTrader enforces the amount of long and short positions to be the same, which creates an unrealistic

TABLE III
TIME SPAN FOR ROLLING WINDOW ANALYSIS

Window No.	Training		Validation		Testing	
	Start date	End date	Start date	End date	Start date	End date
1	2009-01	2018-01	2018-01	2019-01	2019-01	2020-01
2	2009-01	2019-01	2019-01	2020-01	2020-01	2021-01
3	2009-01	2020-01	2020-01	2021-01	2021-01	2022-01
4	2009-01	2021-01	2021-01	2022-01	2022-01	2023-01

TABLE IV

NUMERICAL RESULTS OF FOUR AGENTS USING PPO IN ROLLING WINDOW ANALYSIS. DJI, FINRL, AND MARGIN TRADER OPERATE LONG POSITION ONLY, AND DEEP TRADER AND MARGIN TRADER (L&S) INVOLVES BOTH LONG AND SHORT POSITIONS. THE BEST PERFORMANCE IN EACH ROLLING WINDOW IS MARKED IN BOLD. THE FIRST FOUR METRICS ARE EXPRESSED AS PERCENTAGES.

Window No.	Agent	Accumulated Return	Annual Return	Annual Volatility	Maximum Drawdown	Sharpe Ratio	Calmar Ratio
1	DJI	22.24%	22.24%	12.46%	-6.91%	1.681	3.220
	FinRL	24.74%	24.85%	13.04%	-10.34%	1.774	2.403
	Deep Trader	-2.28%	1.10%	26.40%	-22.74%	0.042	0.049
	Margin Trader (L&S)	24.39%	24.50%	14.28%	-7.97%	1.613	3.074
	Margin Trader	53.70%	53.97%	26.69%	-12.88%	1.758	4.191
2	DJI	6.02%	5.99%	36.81%	-37.09%	0.344	0.162
	FinRL	12.50%	12.50%	29.09%	-28.30%	0.553	0.442
	Deep Trader	-9.76%	-0.96%	42.80%	-44.66%	-0.022	-0.022
	Margin Trader (L&S)	8.98%	8.98%	21.51%	-15.26%	0.509	0.588
	Margin Trader	18.09%	18.09%	36.21%	-28.28%	0.644	0.640
3	DJI	20.23%	20.23%	12.38%	-6.62%	1.556	3.058
	FinRL	19.92%	20.01%	15.69%	-6.85%	1.246	2.922
	Deep Trader	-24.03%	-22.04%	33.10%	-48.51%	0.666	0.454
	Margin Trader (L&S)	24.23%	24.34%	16.91%	-8.38%	1.379	2.905
	Margin Trader	51.03%	51.28%	28.14%	-14.29%	1.619	3.588
4	DJI	-9.40%	-9.43%	19.85%	-21.94%	-0.402	-0.430
	FinRL	4.25%	4.29%	18.83%	-15.46%	0.318	0.277
	Deep Trader	-6.84%	1.32%	41.10%	-31.59%	0.032	0.042
	Margin Trader (L&S)	8.06%	8.12%	19.84%	-19.67%	0.494	0.413
	Margin Trader	9.64%	9.72%	21.91%	-19.82%	0.534	0.491

setting and results in instability, extreme volatility, and large drawdowns.

F. Rolling Window Evaluation

To fully assess the generalization ability and robustness of Margin Trader, we apply rolling-based testing periods, a crucial methodology often lacking in existing works for portfolio management evaluation. Specifically, the training, validation, and testing periods are rolled forward to demonstrate the performance when Margin Trader and baselines consistently learn from recent past data and are deployed for near-future portfolio management. The time span is shown in the Table III, where test periods describe various market conditions:

- 1) Rolling window 1 reflects a bullish market with fluctuations
- 2) Rolling window 2 encompasses the dramatic market downturn during COVID-19 pandemic
- 3) Rolling window 3 represents the abnormal recovery with economic stimulus package
- 4) Rolling window 4 depicts the market impacted by interest-rate rises

Among four RL algorithms, we choose PPO due to its superior performance and exclusively apply it to both FinRL and Margin Trader. Table IV displays numerical results:

- 1) During market upswings, as observed in rolling windows 1 and 3, Margin Trader employing only long positions consistently yields the highest annual return of over 50%, more than double that of other agents. Its equity curve notably surpasses others. Despite coming with the highest volatility and severe MDD, the robust risk-adjusted metrics indicate that the returns are proportionally higher, making the risk potentially worthwhile. Margin Trader engaging in both long and short positions is competitive with FinRL's long-only approach, offering similar returns with slightly better drawdown control. Meanwhile, DJIA earns the least profit, yet maintains the highest stability, reflecting its role as a representative of the market as a whole and indicating a lower-risk investment. DeepTrader, however, fails to capitalize on the upswing: in Window 1 it posts the lowest accumulated return and annual return despite elevated annual volatility, suffers the deepest maximum drawdown, and delivers negligible risk-adjusted performance. A similar pattern is observed in Window 3, suggesting weak trend capture and instability.
- 2) During market distress, such as the COVID pandemic impact around March 2020 in rolling window 2, and the raised interest rates in the first half of 2022 in window 4, short positions in Margin Trader effectively hedge against downturns, leading to higher equity compared to long-only

strategies. As the market rebounded after the COVID crash and during the latter half of 2022, Margin Trader with long positions earned profit with strong resilience. Meanwhile, Margin Trader (Long & Short) maintained some short exposure for risk management, which limited its gains and caused a slight drop in overall equity. This consistent pattern across both periods highlights the robust risk-hedging capability of Margin Trader on short positions during market downturns.

- 3) Performances of portfolio management vary notably across distinct time windows. 2019 (window 1) appears to be the best-performing period for most strategies, characterized by robust equity growth and swift recovery from drawdowns. In contrast, 2022 (window 4) presents significant challenges across all strategies, with DJI even yielding negative returns. This suggests the critical importance of adeptly selecting agents and positions to balance profit generation and risk mitigation. DJI exhibits more moderate returns and minimal risk. FinRL operating long positions shows a middle ground in terms of returns and volatility, generally mirroring market trends. The margin accounts in Margin Trader can lead to substantial returns during favorable market movements, and the short positions offer protection against significant losses in volatile markets, allowing traders to capitalize on both upward and downward market shifts. It is more suitable for investors with a high-risk tolerance. DeepTrader keeps struggling in this challenging setting, failing to converge to a competitive strategy.

VI. CONCLUSION

We present Margin Trader, a novel RL framework specifically designed for margin trading in the stock market. Unlike traditional approaches that primarily focus on cash-only trading, Margin Trader incorporates margin accounts and constraints, offering a more realistic environment for portfolio management. The flexible framework empowers traders to customize crucial settings and caters to specific needs.

Our experiments establish a benchmark example, comparing Margin Trader's performance under long-only positions and an equal allocation between long and short positions. It is evaluated across various DRL algorithms, rolling testing periods, and compared with popular benchmarks. Extensive results demonstrate its ability to generate profitable trading strategies while effectively managing risk under various market conditions. We believe that investors can further adjust the settings to better leverage margin and short positions and optimize their trading strategies based on market outlook and risk preferences.

REFERENCES

- [1] J. Lussange, I. Lazarevich, S. Bourgeois-Gironde, S. Palminteri, and B. Gutkin, "Modelling stock markets by multi-agent reinforcement learning," *Comput. Econ.*, vol. 57, pp. 113–147, 2021.
- [2] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," *Math. Finance*, vol. 33, no. 3, pp. 437–503, 2023.
- [3] Q. Ding, S. Wu, H. Sun, J. Guo, and J. Guo, "Hierarchical multi-scale gaussian transformer for stock movement prediction," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 4640–4646.
- [4] J. Gu, F. P. Deek, and G. Wang, "Stock broad-index trend patterns learning via domain knowledge informed generative network," 2023, *arXiv:2302.14164*.
- [5] J. Gu, J. Ye, A. Uddin, and G. Wang, "DySTAGE: Dynamic graph representation learning for asset pricing via spatio-temporal attention and graph encodings," in *Proc. 5th ACM Int. Conf. AI Finance*, 2024, pp. 388–396.
- [6] J. Gu, S. Shukla, J. Ye, A. Uddin, and G. Wang, "Deep learning model with sentiment score and weekend effect in stock price prediction," *SN Bus. Econ.*, vol. 3, no. 7, 2023, Art. no. 119.
- [7] J. Gu, W. Du, and G. Wang, "RAGIC: Risk-aware generative framework for stock interval construction," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 4, pp. 2085–2096, Apr. 2025.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [9] S. Sun, R. Wang, and B. An, "Reinforcement learning for quantitative trading," *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 3, pp. 1–29, 2023.
- [10] S. Almahdi and S. Y. Yang, "An adaptive portfolio trading system: A risk-return portfolio optimization using recurrent reinforcement learning with expected maximum drawdown," *Expert Syst. Appl.*, vol. 87, pp. 267–279, 2017.
- [11] A. Millea, "Deep reinforcement learning for trading—A critical survey," *Data*, vol. 6, no. 11, 2021, Art. no. 119.
- [12] Y. Zhang, P. Zhao, Q. Wu, B. Li, J. Huang, and M. Tan, "Cost-sensitive portfolio selection via deep reinforcement learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 236–248, Jan. 2020.
- [13] S. Sun et al., "DeepScalper: A risk-aware reinforcement learning framework to capture fleeting intraday trading opportunities," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1858–1867.
- [14] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: An ensemble strategy," in *Proc. 1st ACM Int. Conf. AI Finance*, 2020, pp. 1–8.
- [15] Y. Li, P. Ni, and V. Chang, "Application of deep reinforcement learning in stock trading strategies and stock forecasting," *Computing*, vol. 102, pp. 1305–1322, 2020.
- [16] J. Gu, J. Ye, G. Wang, and W. Yin, "Adaptive and explainable margin trading via large language models on portfolio management," in *Proc. 5th ACM Int. Conf. AI Finance*, 2024, pp. 248–256.
- [17] J. Ye, B. Goswami, J. Gu, A. Uddin, and G. Wang, "From factor models to deep learning: Machine learning in reshaping empirical asset pricing," 2024, *arXiv:2403.06779*.
- [18] E. Dudley and M. Nimalendran, "Margins and hedge fund contagion," *J. Financial Quantitative Anal.*, vol. 46, no. 5, pp. 1227–1257, 2011.
- [19] P. Fortune et al., "Margin requirements, margin loans, and margin rates: Practice and principles," *New England Econ. Rev.*, pp. 19–44, 2000.
- [20] Z. Wang, B. Huang, S. Tu, K. Zhang, and L. Xu, "DeepTrader: A deep reinforcement learning approach for risk-return balanced portfolio management with market conditions embedding," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 643–650.
- [21] G. Huang, X. Zhou, and Q. Song, "Deep reinforcement learning for long-short portfolio optimization," *Comput. Econ.*, pp. 1–37, 2025.
- [22] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," 2017, *arXiv:1706.10059*.
- [23] J. Wang, Y. Zhang, K. Tang, J. Wu, and Z. Xiong, "AlphaStock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1900–1908.
- [24] R. Wang, H. Wei, B. An, Z. Feng, and J. Yao, "Commission fee is not enough: A hierarchical reinforced framework for portfolio management," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 1, 2021, pp. 626–633.
- [25] M. Yang, X. Zheng, Q. Liang, B. Han, and M. Zhu, "A smart trader for portfolio management based on normalizing flows," in *Proc. IJCAI Conf. Artif. Intell.*, 2022, pp. 4014–4021.
- [26] Y. Liu, Q. Liu, H. Zhao, Z. Pan, and C. Liu, "Adaptive quantitative trading: An imitative deep reinforcement learning approach," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 2128–2135.
- [27] H. Niu, S. Li, and J. Li, "MetaTrader: An reinforcement learning approach integrating diverse policies for portfolio optimization," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1573–1583.
- [28] K. Xu, Y. Zhang, D. Ye, P. Zhao, and M. Tan, "Relation-aware transformer for portfolio policy learning," in *Proc. 29th Int. Conf. Int. Joint Conferences Artif. Intell.*, 2021, pp. 4647–4653.

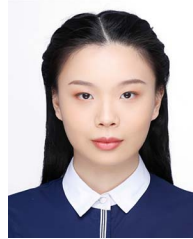
- [29] X.-Y. Liu et al., "FinRL-meta: Market environments and benchmarks for data-driven financial reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 1835–1849.
- [30] X.-Y. Liu, H. Yang, J. Gao, and C. D. Wang, "FinRL: Deep reinforcement learning framework to automate trading in quantitative finance," in *Proc. 2nd ACM Int. Conf. AI Finance*, 2021, pp. 1–9.
- [31] W. F. Sharpe, "The sharpe ratio," *Streetwise—Best J. Portfolio Manage.*, vol. 3, pp. 169–85, 1998.
- [32] E. Vittori, M. Trapletti, and M. Restelli, "Option hedging with risk averse reinforcement learning," in *Proc. 1st ACM Int. Conf. AI Finance*, 2020, pp. 1–8.
- [33] R. A. de Oliveira, H. S. Ramos, D. H. Dalip, and A. C. M. Pereira, "A tabular sarsa-based stock market agent," in *Proc. 1st ACM Int. Conf. AI Finance*, 2020, pp. 1–8.
- [34] G. Jeong and H. Y. Kim, "Improving financial trading decisions using deep Q-learning: Predicting the number of shares, action strategies, and transfer learning," *Expert Syst. Appl.*, vol. 117, pp. 125–138, 2019.
- [35] Z. Xiong, X.-Y. Liu, S. Zhong, H. B. Yang, and A. Walid, "Practical deep reinforcement learning approach for stock trading," 2018, *arXiv:1811.07522*.
- [36] Y. Ye et al., "Reinforcement-learning based portfolio management with augmented asset movement prediction states," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 1112–1119.
- [37] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [39] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [40] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [41] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 1, pp. 1–8, 2021.



Jingyi Gu received the BE degree in economics statistics from Xiamen University, China, in 2017, the MS degree in data science from Rutgers University, New Brunswick, in 2019, and the PhD degree in computer science from the New Jersey Institute of Technology, in 2025. She is currently an applied scientist with Amazon. Her research interests include deep learning applications in various fields.



Wenlu Du received the PhD degree in computer science from the New Jersey Institute of Technology, in 2024. She was a software engineer with the Alpine Electronics of America. She is currently an assistant professor with Computer Science Department, Skidmore College. Her research interests include reinforcement learning and its applications in transportation and finance.



Xinyun Zhao received the MS degree in computer science from New York University, New York, in 2020. She was a software engineer for two years. Her research interests include reinforcement learning applications in various fields.



A M Muntasir Rahman received the bachelor's degree in computer science from BRAC University, Dhaka, Bangladesh, in 2018, and the PhD degree in computer science from the New Jersey Institute of Technology. He is currently a postdoctoral researcher with Rutgers University, New Brunswick. His research focuses on the intersection of large language models and natural language processing.



Guiling (Grace) Wang (Fellow, IEEE) received the PhD degree. She is currently a distinguished professor of computer science and associate dean for Research with the NJIT's Ying Wu College of Computing. She is also the CFA. As the founding director of NJIT's AI Center for Research, she leverages her AI expertise to address challenges in fields like Finance and Transportation. Her pioneering research in deep reinforcement learning for traffic light optimization earned the IEEE Vehicular Technology Society's 2023 Best Paper Award, and her project was one

of four nationally funded by the U.S. Department of Transportation's FHWA Exploratory Advanced Research (EAR) program in 2020, selected from 122 submissions nationwide. Prof. Wang has served as Lead Program Co-Chair for the ACM Conference on AI in Finance (2023) and General Co-Chair (2024) and is an associate editor for prominent journals such as *ACM Computing Surveys* and *IEEE Transactions on Knowledge and Data Engineering*. Beyond academia, she serves as a member of the New Jersey Supreme Court's Committee on AI and the Courts, contributes to the Governor's AI Taskforce, and acts as a subject matter expert in AI for the U.S. Department of Homeland Security's SAGE program.